Deadlock

Introduction

- Computer systems have resources that can be used by one process at a time.
- Many applications needs access to more than one resource.
- Deadlocks can occur on hardware resources or software resources.

1. One computer with many applications where processes need exclusive access to not one resource bet several. For example, two processes each want to record a scanned document on a Blu-ray disc. Process A requests permission to use the scanner and is granted it. Process B is programmed differently and requests the Blue-ray recorder first and is also granted it. Now A asks for the Blu-ray recorder, but the request is suspended until

B releases it. Unfortunately, instead of releasing the Blu-ray recorder, B asks for the scanner. At this point. Both processes are blocked and will remain so forever.

2. Across machines – Many offices have local areas network with many computers connected to it. Often devices such as scanners, Blu-ray/DVD recorders, printers, and tape drivers are connected to the network as shared resources, available to any user on any machine. If these devices can be reserved remotely (i.e. from the user's home machine), deadlocks of the same kind can occur as described above. More complicated situations can cause deadlocks involving three, four, or more devices and users.

3. In database systems – A program may have to lock several records it is using, to avoid race conditions. If process A locks record R1 and process B locks record R2 and then each process tries to lock the other one's record, we also have a deadlock.

What Is Deadlock In Operating System

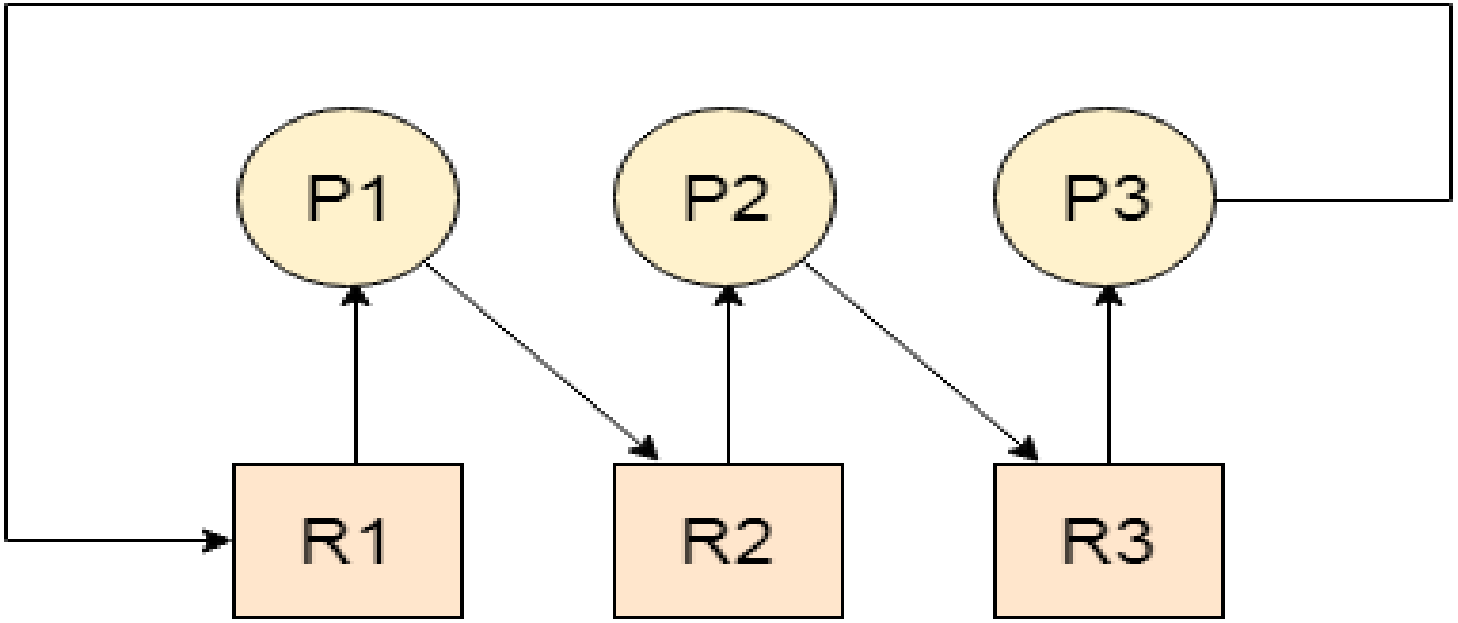
- Every process needs some resources to complete its execution. However, the resource is granted in a sequential order.
  - The process request for some resources.
  - OS grants the resource if it is available otherwise let the process wait.
  - The process uses it and releases it on the completion.

"A Deadlock is a situation where each of the computer process waits for a resource which is being assigned to some another process. In this situation, none of the process gets executed since the resource it needs, is held by some other process which is also waiting for some other resource to be released".

- Lets assume that there are three processes P1,P2 and P3 and 3 resources R1,R2 and R3.
  - R1 is assigned to P1
  - R2 is assigned to P2
  - R3 is assigned to P3

- After some time P1 demands for R2 which is being used by P2.P1 halts its execution since it can't complete without R2.

- P2 also demands for R3 which is being used by P3.P2 also stops its execution because it can’t continue without R3.
- P3 also demands for R1 which is being used by P1 therefore P3 also stops its execution.
- In this scenario, a cycle is being formed by the three processes.
- None of the process is progressing and they are all waiting.
- The computer became unresponsive since all the processes got blocked.



Difference Between Starvation And Deadlock

Deadlock	Starvation
Deadlock is a situation where no process got blocked and no process proceed.	Low-priority processes got blocked and high-priority processes proceeded.
Deadlock is and infinite waiting	Starvation is a long waiting, not infinite
Every Deadlock is always a starvation.	Every starvation need not be deadlock.
The requested resource is blocked by the other process.	The requested resource is continuously be used by the higher priority processes.
Deadlock happens when Mutual exclusion, hold and wait, No preemption and circular wait occurs simultaneously.	It occurs due to uncontrolled priority and resource management.

Necessary Conditions For Deadlocks

Deadlock occurs when all the 4 conditions happen simultaneously.

1. Mutual Exclusion
   - Two processes cannot use the same resource at the same time.
2. Hold and wait
   - Process waits for some resources while holding another resource at the same time.
3. No preemption
   - The process which once scheduled will be executed till the completion. No other process can be scheduled by the scheduler meanwhile
4. Circular wait
   - All the processes must be waiting for the resources in a cyclic manner so that the last process is waiting for the resource which is being held by the first process.

Strategies For Handling Deadlock

1. Deadlock ignorance
2. Deadlock prevention
3. Deadlock avoidance
4. Deadlock detection and recovery

Deadlock Ignorance

- In this approach OS assumes deadlock never occurs and it simply ignores deadlock.
- This approach is best suitable for a single end-user system where user uses the system only for browsing and all other normal stuff.
- In these types of systems, the user has to simply restart the computer in the case of deadlock.
- Windows and Linux are mainly using this approach.

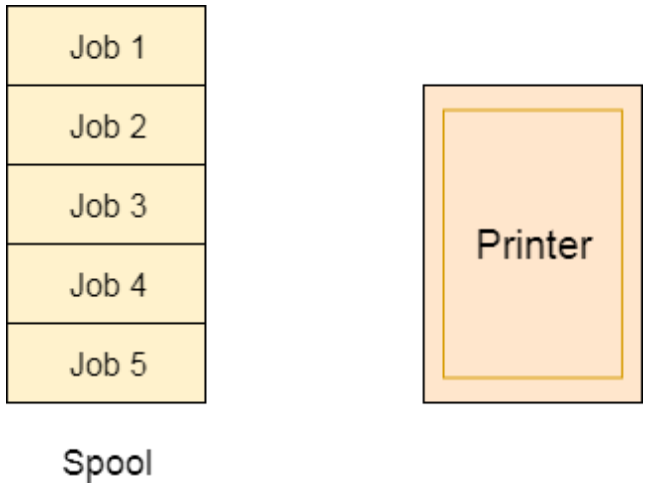
Deadlock Prevention

- Mutual exclusion, hold and wait, no preemption, and circular wait happening simultaneously cause the deadlock.
- If one condition is violated we can prevent happening of the deadlock.

Violating mutual exclusion

- Spooling is one suggested method to overcome mutual exclusion. Spooling can apply to a resource like a printer.
- There is a memory associated with the printer which stores jobs from each of the process into it.
- Later, Printer collects all the jobs and print each one of them according to FCFS.
- By using this mechanism, the process doesn't have to wait for the printer and it can continue whatever it was doing.

- Later, it collects the output when it is produced.



- Spooling suffers from two kinds of problems.
  - This cannot be applied to every resource.
  - After some time, race conditions arise between processes to get space in that spool.
- We cannot force a resource to be used by more than one process at the same time since it will not be fair enough and some serious problems may arise in the performance.

- Therefore, we cannot violate mutual exclusion for a process practically.

Violation of Hold and wait

- Hold and wait condition lies when a process holds a resource and waits for some other resource to complete its task.
- This can be implemented practically if a process declares all the resources initially.
- But the computer system can’t determine the necessary resources initially.
- The problem with the approach is:
    1) Practically not possible.
    2) Possibility of getting starved will increase due to the fact that some processes may hold a resource for a very long time.

No preemption

- Deadlock arises due to the fact that a process can't be stopped once it starts.
- However, if we take the resource away from the process which is causing deadlock then we can prevent deadlock.
- This is not a good approach since if we take a resource away which is being used by the process then all the work which it has done till now can become inconsistent.

Circular wait

- To violate circular wait, we can assign a priority number to each of the resources.
- A process can't request a lesser priority resource.

- This ensures that not a single process can request a resource that is being utilized by some other process and no cycle will be formed.
- Among all the methods, violating Circular wait is the only approach that can be implemented practically.

Deadlock avoidance

- In deadlock avoidance, the request for any resource will be granted if the resulting state of the system doesn't cause deadlock in the system.
- The state of the system will continuously be checked for safe and unsafe states.

- In order to avoid deadlocks, the process must tell OS, the maximum number of resources a process can request to complete its execution.
- Process should declare the maximum number of resources of each type it may ever need.

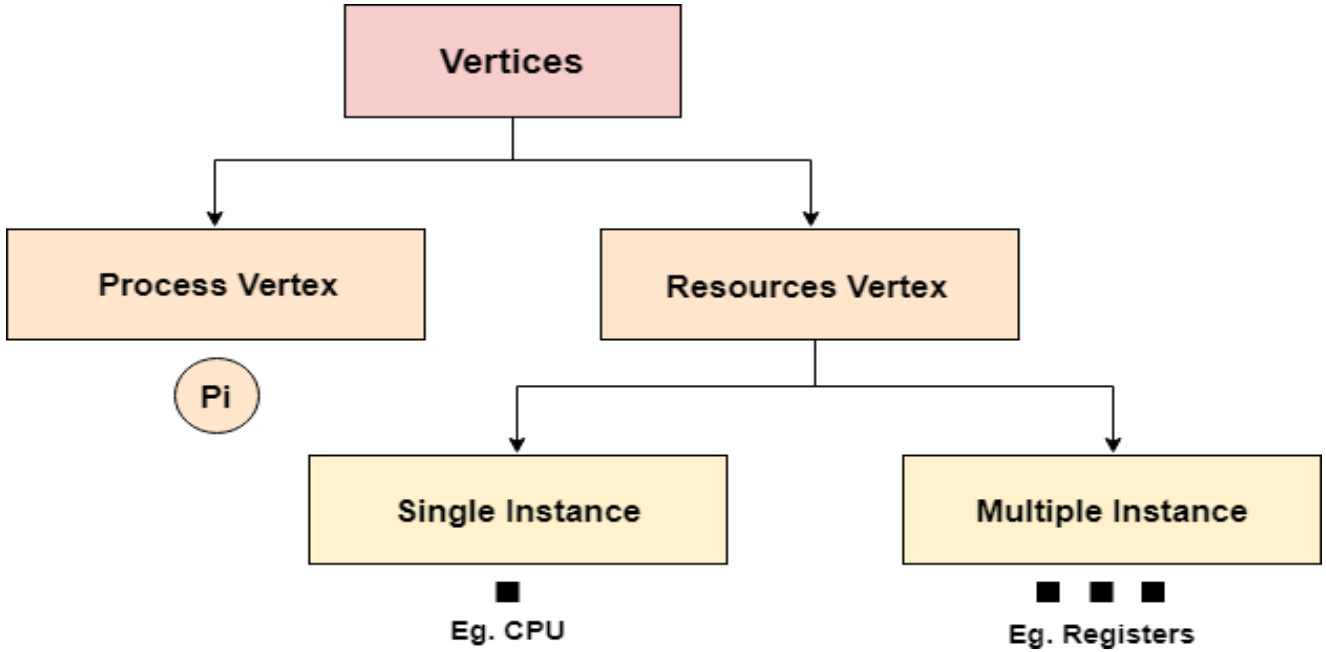
Safe and Unsafe states

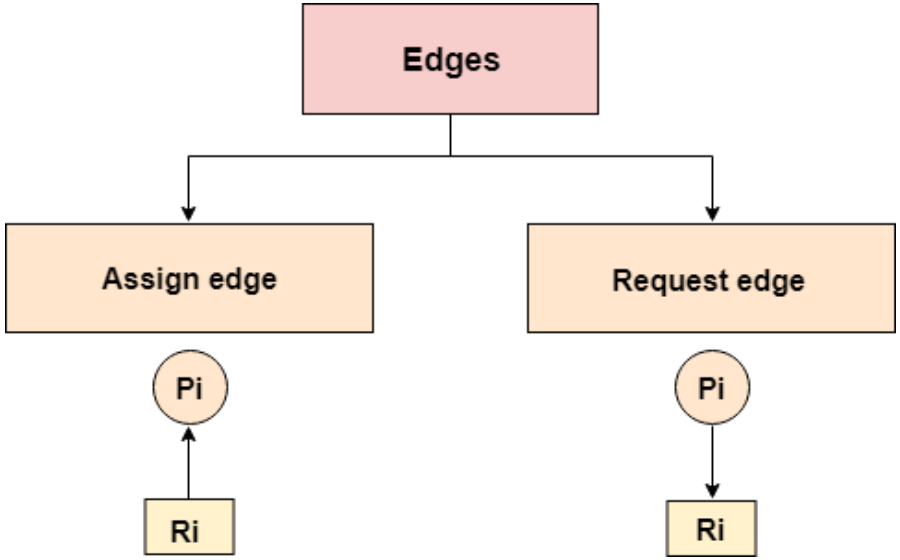
- Safe state-> No deadlock
- Unsafe state-> Possibility of deadlock

- Avoidance->Assuring that system never enters in unsafe state.

Resource Allocation Graph(RAG)

- Resource allocation graph is the complete information about all the processes that are holding some resources or waiting for some resources.
- In the Resource allocation graph, the process is represented by a Circle while the Resource is represented by a rectangle.
- Vertices are mainly of two types, Resource and process. Each of them will be represented by a different shape.
- The circle represents process while rectangle represents resource.

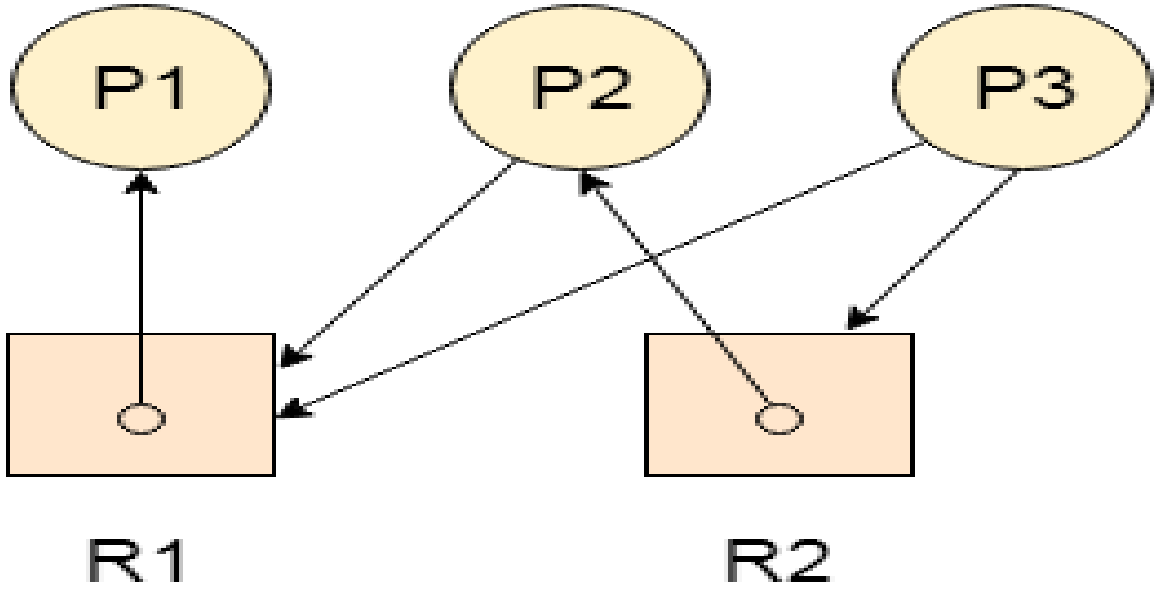

Vertices
Process Vertex
Resources Vertex
Pi
Single Instance
Multiple Instance
Eg. CPU
Eg. Registers


Edges
Assign edge
Request edge
Pi
Pi
Ri
Ri

- A resource can have more than one instance. Each instance will be represented by a dot inside the rectangle.
- Edges in RAG are also of two types, one represents assignment and other represents the wait of a process for a resource.
- A resource is shown as assigned to a process if the tail of the arrow is attached to an instance to the resource and the head is attached to a process.
- A process is shown as waiting for a resource if the tail of an arrow is attached to the process while the head is pointing towards the resource.

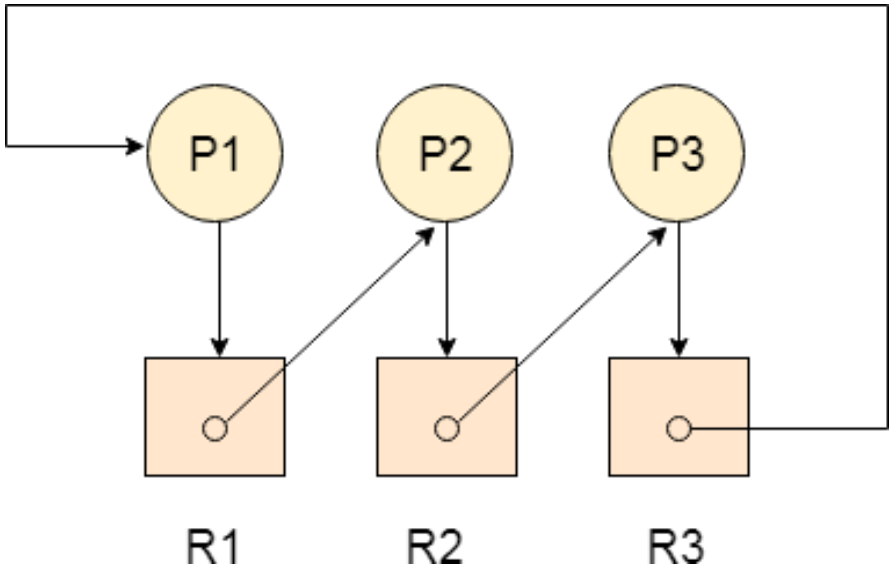
Example

- Let's consider 3 processes P1, P2 and P3, and two types of resources R1 and R2. The resources are having 1 instance each.
- According to the graph, R1 is being used by P1, P2 is holding R2 and waiting for R1, P3 is waiting for R1 as well as R2.
- The graph is deadlock free since no cycle is being formed in the graph.



Deadlock Detection Using RAG

- If a cycle is being formed in a Resource allocation graph where all the resources have the single instance then the system is deadlocked.
- In Case of Resource allocation graph with multi-instanced resource types, Cycle is a necessary condition of deadlock but not the sufficient condition.



- **The before example** contains three processes P1, P2, P3 and three resources R2, R2, R3. All the resources are having single instances each.
- If we analyze the graph then we can find out that there is a cycle formed in the graph since the system is satisfying all the four conditions of deadlock.

Allocation Matrix

- Allocation matrix can be formed by using the Resource allocation graph of a system. In Allocation matrix, an entry will be made for each of the resource assigned. For Example, in the following matrix, en entry is being made in front of P1 and below R3 since R3 is assigned to P1.

Process	R1	R2	R3
P1	0	0	1
P2	1	0	0
P3	0	1	0

Request Matrix

- In request matrix, an entry will be made for each of the resource requested. As in the following example, P1 needs R1 therefore an entry is being made in front of P1 and below R1.

Process	R1	R2	R3
P1	1	0	0
P2	0	1	0
P3	0	0	1

- Neither we are having any resource available in the system nor a process going to release.
- Each of the process needs at least single resource to complete therefore they will continuously be holding each one of them.
- We cannot fulfill the demand of at least one process using the available resources therefore the system is deadlocked

single resource instance
=printers/Scanners/magnetic tape drivers

Processes	Max need	Current allocation	Need
P0	10	5	5
P1	4	2	2
P2	9	2	7

max need=current allocation+Need

Available=3

p0 5>3
p1 2<=3 can execute
available=3+2=5
p2 7>5
p0 5<=5 can execute
available=5+5=10
p2 7<=10 can execute
available=10+2=12

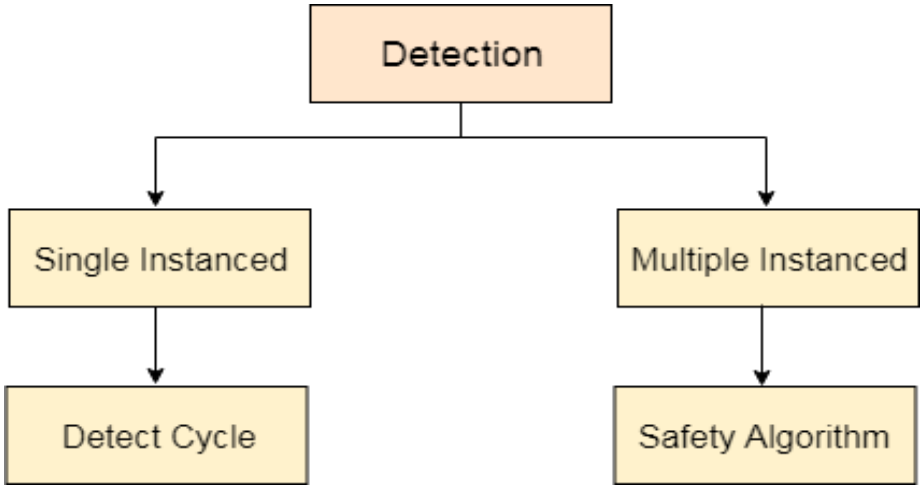
sequence of execution=p1,p0,p2

if need<=Available
exceute the process

if need>Available
simply check for the next process

Deadlock Detection And Recovery

- System does not attempt to prevent deadlock from occurring. Instead it lets them occur tries to detect when this happens and take actions to recover after that fact.



- In single instanced resource types, if a cycle is being formed in the system then there will definitely be a deadlock.
- On the other hand, in multiple instanced resource type graph, detecting a cycle is not just enough.

* Multiple instance → Banker's algorithms

	Allocation				Max				Available				Need			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
P_0	0	0	1	2	0	0	1	2	~~1~~	~~5~~	~~2~~	~~0~~	0	0	0	0
P_1	1	0	0	0	1	7	5	0	~~1~~	~~5~~	~~3~~	~~2~~	0	7	5	0
P_2	1	3	5	4	2	3	5	6	~~2~~	~~8~~	~~8~~	~~6~~	1	0	0	2
P_3	0	6	3	2	0	6	5	2	~~2~~	~~14~~	~~11~~	~~8~~	0	0	2	0
P_4	0	0	1	4	0	6	5	6	2	14	12	12	0	6	4	2

Need = Max - Allocation

safe sequence

safe sequence = $P_0, P_3, \dots$

= $P_3, P_0, \dots$

safe sequence = P_0, P_2, P_3, P_4, P_1

1) P_0, 0000 ✓

Available = 1520 + ~~0000~~ 0012

= 1532

2) P_1, 1532 < 0750 ✗

3) P_2, 1532 > 1002 ✓

Available = 1532 + 1354

= 2886

4) P_3, 2886 > 0020 ✓

Available = 2886 + 0632

= 2 14 11 8

5) P_4, 2 14 11 8 > 0642 ✓

Available = 2 14 11 8 + 0014

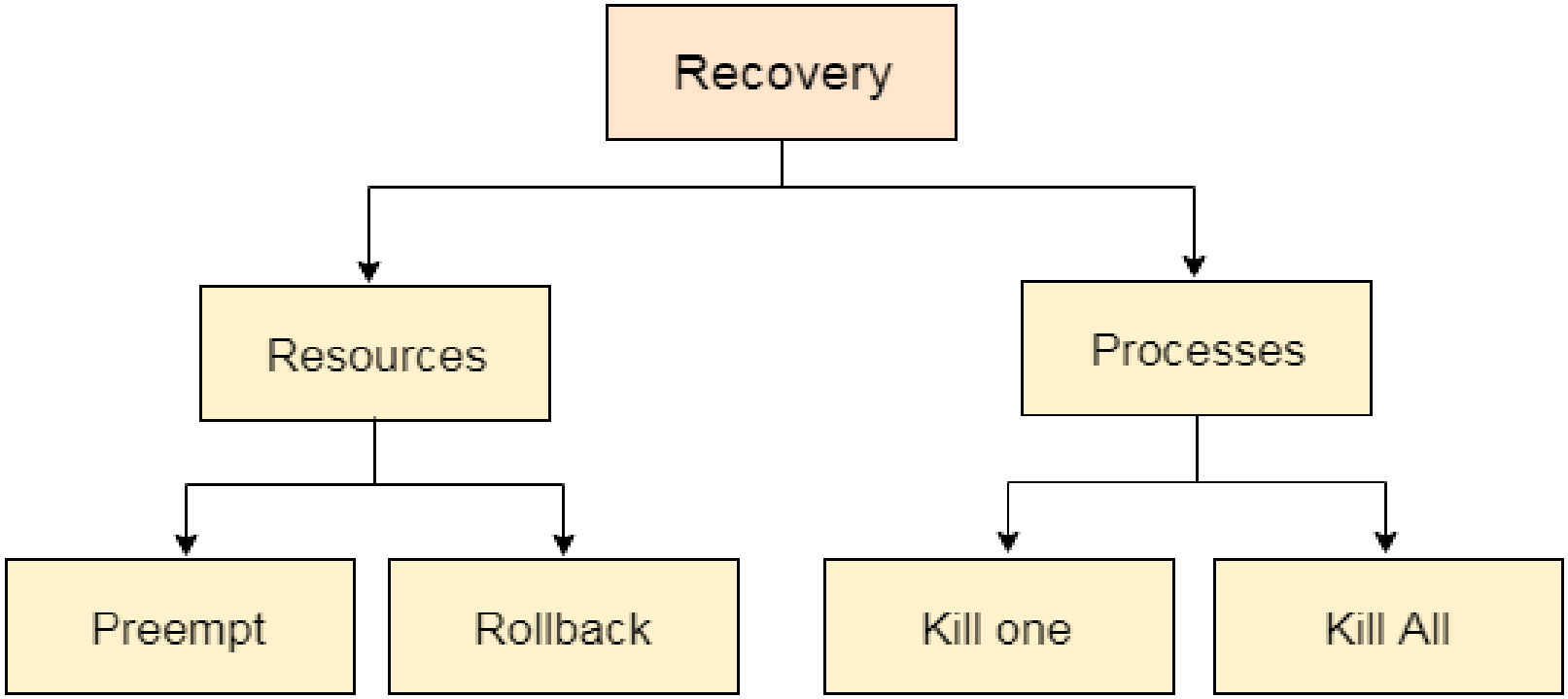
= 2 14 12 12

6) P_1, 2 14 12 12 > 0750 ✓

Available = 2 14 12 12 + 1000

= 3 14 12 12

- We have to apply the safety algorithm on the system by converting the resource allocation graph into the allocation matrix and request matrix.
- In order to recover the system from deadlocks, either OS considers resources or processes.



- **Preempt the resource**-We can snatch one of the resources from the owner of the resource (process) and give it to the other process with the expectation that it will complete the execution and will release this resource sooner.
- **Rollback to a safety state**-System passes through various states to get into the deadlock state. The operating system can rollback the system to the previous safe state. For this purpose, OS needs to implement check pointing at every state.
- **Kill a process**-Generally, Operating system kills a process which has done least amount of work until now.
- **Kill all processes**-Killing all process will lead to inefficiency in the system because all the processes will execute again from starting.